Week 2 Project: Advanced Line Calculations

Thermal Analysis, Voltage Drop, and Conductor Sizing

Enrique Antonio Raudales Rodriguez

October 18, 2025

University of Almería
Medium and Low Voltage Electrical Installations
Academic Year 2025-2026

Contents

1 Introduction

This report details the analysis of an insulated electrical conductor's thermal and electrical properties under various operational scenarios. The primary objective is to determine the conductor's maximum current-carrying capacity (ampacity) while respecting its maximum allowable operating temperature. The analysis is performed using a suite of MATLAB scripts that model the physical phenomena of heat generation and dissipation.

The study is divided into three main parts:

- **Thermal Analysis:** Investigating how the conductor heats up under an electrical load and how it dissipates this heat to the surrounding environment. This includes calculating the maximum steady-state current and modeling the transient heating behavior over time .
- **Voltage Drop Analysis:** Making profiles of the voltage at different points in the installation, either in a continous line or across various key points in the electrical distribution from the grid to the final consumer. More precise and simplified formulas are compared
- **Economic Analysis:** Evaluating the lifecycle cost of different standard conductor sizes to identify the most economically optimal choice, balancing the initial investment against the long-term cost of energy losses.

The primary objective is to develop a comprehensive engineering toolkit in MATLAB that moves beyond prescriptive tables to model the underlying physics. This includes determining the maximum admissible current based on thermal equilibrium, utilizing the Blondel formula for accurate voltage drop calculations, and applying Kelvin's Economic Law to find the most cost-effective conductor size over the project's lifecycle. All analyses are conducted using a modular suite of scripts, with the key findings synthesized into this document.

2 Thermal Equilibrium and Heat Transfer Analysis

The core of this analysis is to determine the temperature a conductor reaches under a given electrical load and specific environmental conditions. This is governed by the principle of thermal equilibrium, where the heat generated within the conductor equals the heat dissipated to its surroundings. This section details the mathematical models used to represent this process, as implemented in the MATLAB scripts `E6_TransientHeating.m` and `E7_HeatDissipation.m`.

2.1 The Fundamental Heat Balance Equation

The temperature of the conductor over time is a dynamic process. The rate of change of temperature is proportional to the net power flow (generated heat minus dissipated heat). This relationship is expressed by the following first-order ordinary differential equation (ODE), which forms the basis of the transient analysis in `E6_TransientHeating.m`:

$$m \cdot c_p \frac{dT}{dt} = P_{\text{generated}}(T) - P_{\text{dissipated}}(T) \tag{1}$$

Where:

- m: Mass of the conductor (kg)
- c_p: Specific heat capacity of the conductor material (J/kg°C)
- $\frac{dT}{dt}$: Rate of temperature change over time (°C/s)
- $P_{\text{generated}}(T)$: Heat generated by the Joule effect, dependent on temperature (W)
- $P_{\text{dissipated}}(T)$: Heat dissipated to the environment, dependent on temperature (W)

Thermal equilibrium is the steady-state condition where the temperature stops changing, i.e., $\frac{dT}{dt} = 0$. At this point, the heat generation is perfectly balanced by the heat dissipation:

$$P_{\text{generated}}(T_{\text{eq}}) = P_{\text{dissipated}}(T_{\text{eq}}) \tag{2}$$

The script `E4_ThermalEquilibrium.m` solves this equation to find the equilibrium temperature, T_{eq}.

2.2 Heat Generation ($P_{\text{generated}}$)

Heat is generated in the conductor due to the flow of electric current (Joule's Law). The power generated is a function of the current and the conductor's electrical resistance. Since resistance itself is a function of temperature, the heat generation term is temperature-dependent.

$$P_{\text{generated}}(T) = I^2 \cdot R(T) \tag{3}$$

The resistance at a given temperature T, denoted as $R(T)$, is calculated using a linear approximation based on a reference resistance (R_{20} at 20°C), as implemented in `E3_ResistanceTemperatureCorrection.m`:

$$R(T) = R_{20} \cdot (1 + \alpha \cdot (T - T_{\text{ref}})) \tag{4}$$

Where:

- I: Electrical current (A)
- $R(T)$: AC resistance at temperature T (Ω)
- R_{20}: AC resistance at the reference temperature T_{ref} (Ω)
- α: Temperature coefficient of resistance for the material (1/°C)
- T_{ref}: Reference temperature, typically 20°C

2.3 Heat Dissipation ($P_{\text{dissipated}}$)

The mechanisms for heat dissipation are highly dependent on the installation environment. The script `E7_HeatDissipation.m` acts as a router, selecting the appropriate physical model based on the chosen scenario.

2.3.1 Underground Installation

For a buried cable, heat dissipates primarily through conduction into the surrounding soil. The model calculates dissipation based on the thermal resistance of the soil.

$$P_{\text{diss}} = \frac{T_s - T_{\text{soil}}}{R_{\text{th,soil}}} \tag{5}$$

Where T_s is the cable surface temperature and T_{soil} is the ambient soil temperature. The thermal resistance of the soil, $R_{\text{th,soil}}$, is calculated using an approximation derived from Kennelly's formula:

$$R_{\text{th,soil}} = \frac{\rho_{\text{soil}}}{2\pi L} \ln\left(\frac{4h}{D}\right) \tag{6}$$

Where:

- ρ_{soil}: Thermal resistivity of the soil (K·m/W)
- L: Length of the cable (m)
- h: Burial depth to the center of the cable (m)
- D: Outer diameter of the cable (m)

2.3.2 Overhead Installation

For an overhead line exposed to the atmosphere, the heat dissipation is a combination of convection, radiation, and solar heat gain.

$$P_{\text{diss}} = P_{\text{conv}} + P_{\text{rad}} - P_{\text{solar}} \tag{7}$$

1. Convection (P_{conv}): Heat transfer due to air movement.

$$P_{\text{conv}} = h_c \cdot A_s \cdot (T_s - T_{\text{air}}) \tag{8}$$

The convective heat transfer coefficient, h_c, is derived from the Nusselt number (Nu), which depends on fluid dynamics (Reynolds number Re for forced convection due to wind, and Grashof number Gr for natural convection).

- $A_s = \pi D L$: Surface area of the cable (m^2)
- $h_c = \frac{k_{\text{air}}}{D} Nu$: Convective heat transfer coefficient ($\text{W/m}^2\text{K}$)
- The formulas for Nu are empirical correlations depending on Re and Prandtl number (Pr) for forced convection, or Gr and Pr for natural convection.

2. Radiation (P_{rad}): Heat emitted as electromagnetic waves. This is calculated using the Stefan-Boltzmann law, which relates radiated power to the fourth power of the absolute temperature.

$$P_{\text{rad}} = \varepsilon \cdot \sigma \cdot A_s \cdot (T_{s,K}^4 - T_{\text{air},K}^4) \tag{9}$$

Where:

- ε: Emissivity of the cable surface (dimensionless)

- σ: Stefan-Boltzmann constant (5.67×10^{-8} W/m^{2}K^4)
- $T_{s,K}, T_{\text{air},K}$: Absolute temperatures of the surface and air in Kelvin (K)

3. Solar Gain (P_{solar}): Heat absorbed from solar radiation.

$$P_{\text{solar}} = a \cdot S \cdot D \cdot L \tag{10}$$

Where:

- a: Solar absorptivity of the cable surface (dimensionless)
- S: Solar irradiance (W/m^2)
- $D \cdot L$: Projected area of the cable exposed to the sun (m^2)

2.4 AC Resistance and Maximum Current

For accuracy, the conductor's AC resistance (R_{AC}) is calculated, accounting for skin and proximity effects (`E3b_AC_Resistance_Factor.m`). The maximum admissible current ($I_{\max}$) is the current that causes the conductor to reach its maximum insulation temperature (e.g., 90 °C for XLPE), as determined in `E2_MaximumCurrent.m`.

$$I_{\max} = \sqrt{\frac{P_{\text{dissipated\_max}}}{R_{AC,T_{\max}}}} \tag{11}$$

The AC resistance is calculated by taking the DC Resistance and applying a correction factor for the skin effect and another for the proximity effect. These formulas have been taken from IEC 60287-1-1

$$R_{AC} = R_{DC} \cdot (1 + k_{\text{skin}} + k_{\text{prox}}) \tag{12}$$

The skin effect is caused by the charges habit of collecting near the edge of the conductor, clumping together and increasing the resistance. It is less notable on smaller wires and lower frequencies, and in the domestic setting it is often taken as negligible.

$$k_{\text{skin}} = \frac{x_s^4}{192 + 0.8x_s^4} \tag{13}$$

The proximity effect is due to the electromagnetic radiation leaking from one conductor to other nearby conductors, and is more pronounced the more wires are lumped together

$$k_{\text{prox}} = \frac{x_p^4}{192 + 0.8x_p^4} \left(\frac{d}{s}\right)^2 \left(0.312 \left(\frac{d}{s}\right)^2 + \frac{1.18}{\frac{x_p^4}{192+0.8x_p^4} + 0.27}\right) \tag{14}$$

Parameter xs and xp are the same

$$x_s^2 = \frac{8\pi f}{R'_{DC}} \times 10^{-7} \tag{15}$$

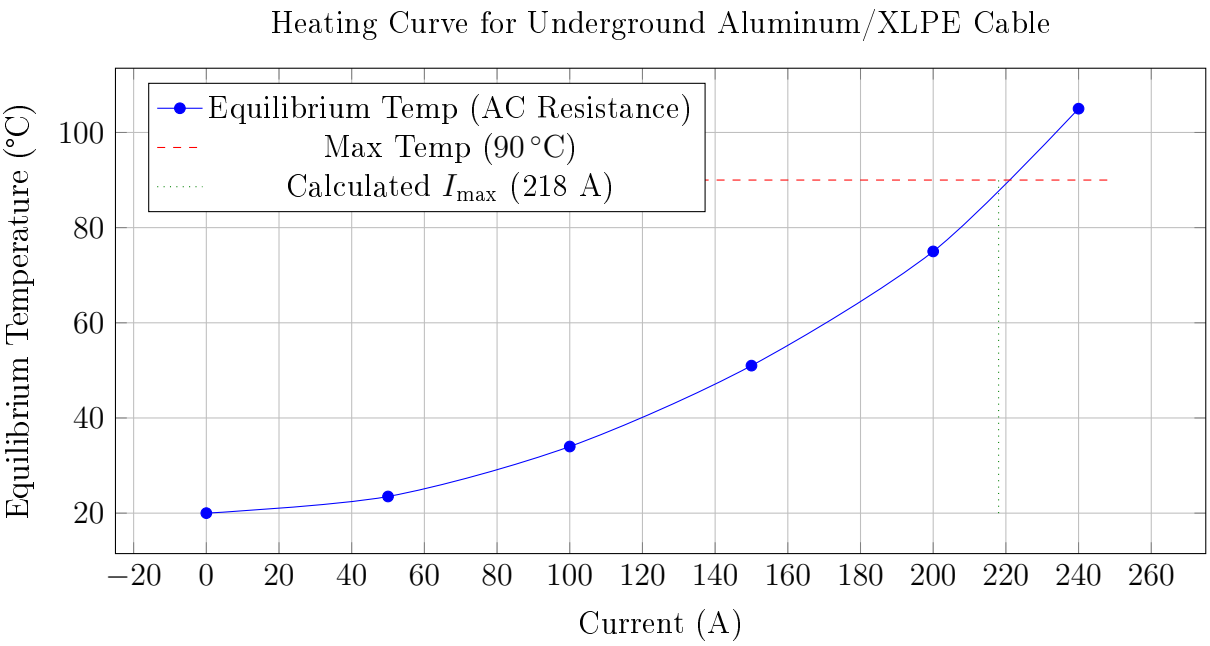


Figure 1: Equilibrium conductor temperature as a function of load current.

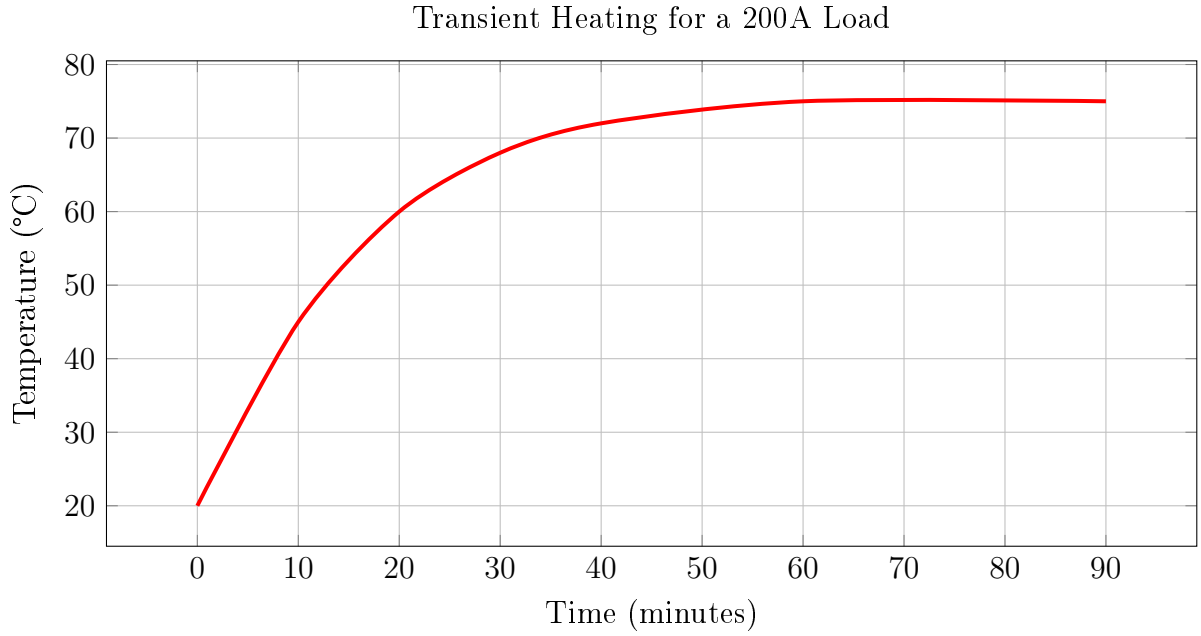


Figure 2: Transient temperature response of the conductor over time.

2.5 Thermal Analysis Results

The charts below, representing the output of the MATLAB analysis, illustrate the conductor's thermal behavior. Figure 1 shows the steady-state temperature for different currents, while Figure 2 shows how temperature evolves over time.

3 Advanced Voltage Drop Analysis

This section details a precise voltage drop analysis, including reactive effects, based on the MATLAB functions prefixed with 'F'.

3.1 Blondel's Formula

The approximate voltage drop in a three-phase AC circuit is calculated using Blondel's formula, which is valid for most well designed circuits where the angle between the voltage at the source and at the load is very small. The script (`F2_calculateExactVoltageDrop.m` is used, which accounts for both resistance (R) and inductive reactance (X_L). Where d is a configuration parameter and takes the value of d = 2 for a single phase line with return and d = 1.732 for three phase lines.

$$\Delta V = \sqrt{3} \cdot I \cdot (R \cos \varphi + X_L \sin \varphi) \tag{16}$$

3.2 REBT Compliance and Voltage Profile

Compliance with the Spanish regulation (REBT) is mandatory. The script `F4b_checkInstallationComp` automates checking the tiered voltage drop limits for a full installation (LGA, DI, etc.).
The script `F6_plotInstallationProfile.m` visualizes this drop, as shown in Figure 3.

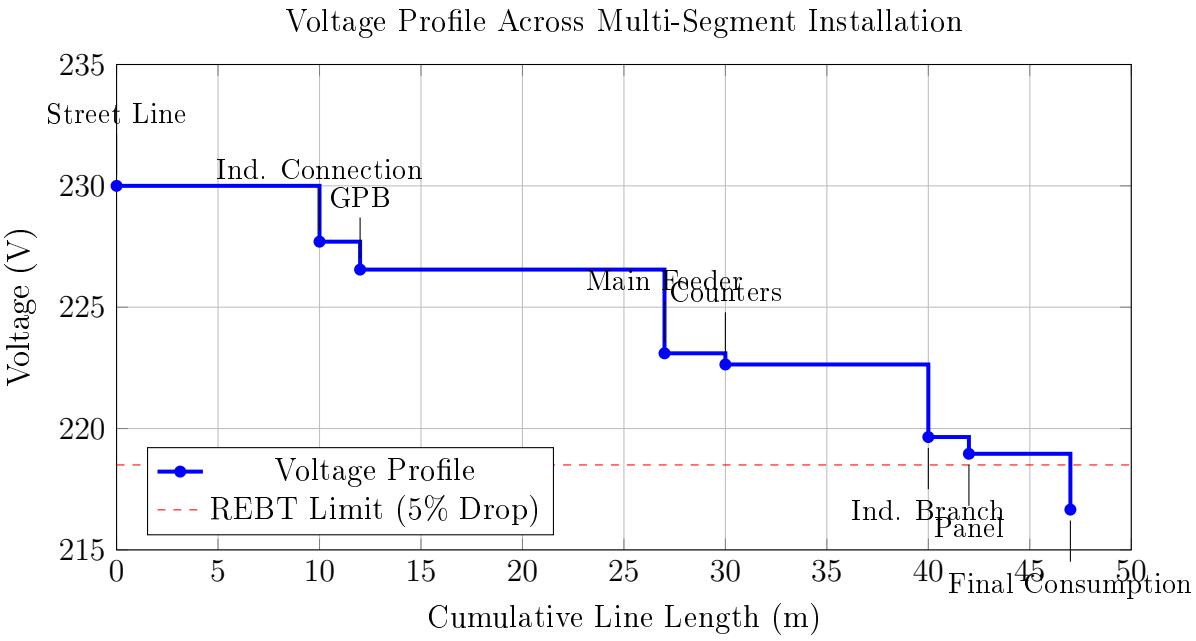
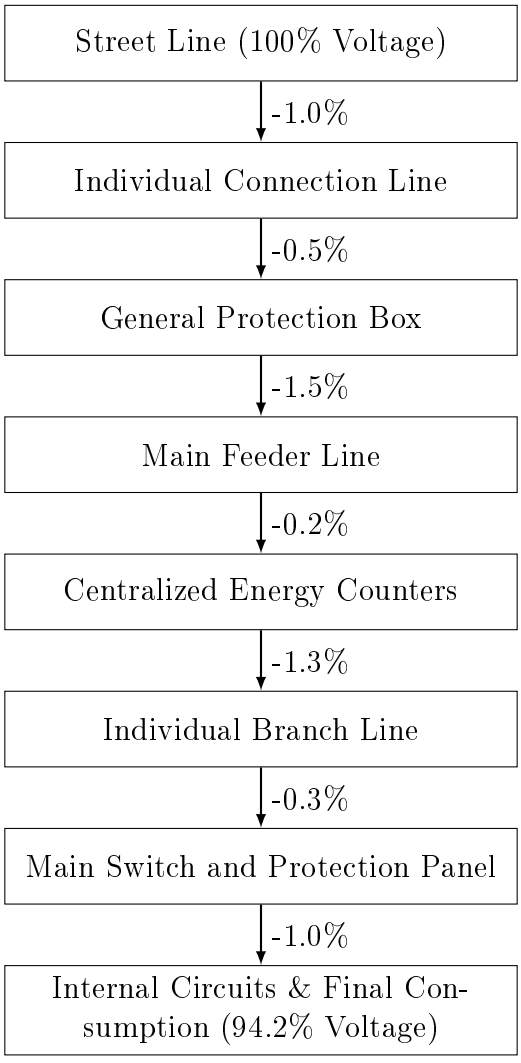


Figure 3: Voltage profile across an installation, showing the drop at each section.



4 Conductor Sizing and Economic Analysis

The final step is to select the optimal conductor cross-section. This analysis, based on the 'G' series of MATLAB scripts, integrates technical requirements with long-term economic factors.

4.1 Technical and Economic Sizing Criteria

Conductor sizing is based on two main criteria:

1. **Technical Minimum:** The smallest standard cross-section that satisfies both the maximum admissible current ($I_{\max}$) and the REBT voltage drop limits. This is calculated via `G2_calculateRequiredSection.m` and `G3_selectStandardSection.m`.

2. **Economic Optimum:** The cross-section that results in the lowest total lifecycle cost (LCC), considering both the initial cable purchase price and the cumulative cost of energy losses (I^2R) over the project's lifespan. This is a simpler, albeit similar approach to Kelvin's Economic Law for calculating the economic conductor size.

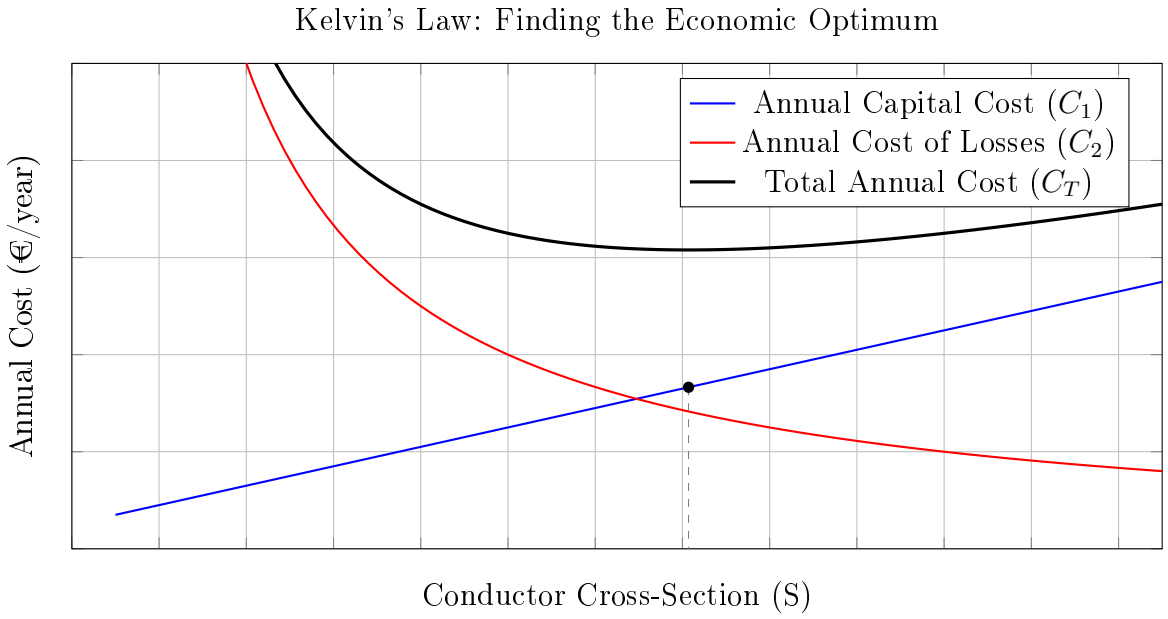


Figure 4: The optimal conductor size (S_{opt}) occurs at the minimum of the total annual cost curve, which corresponds to the point where the cost of losses and the annualized capital cost curves intersect.

4.2 Lifecycle Cost Analysis

The lifecycle cost is modeled in `G4_calculateLifecycleCost.m` as:

$$C_{\text{total}} = C_{\text{initial\_cable}} + C_{\text{energy\_losses}} \tag{17}$$

While a smaller cable is cheaper initially, it has higher resistance and thus a higher cost of energy losses over time. A larger, more expensive cable has lower losses. The optimal size minimizes this sum.

4.3 Sizing Analysis Results

The economic analysis is visualized in Figure 5. The stacked bars show the components of the total cost, while the red line shows the total LCC. The analysis identifies both the smallest technically compliant section and the larger, economically optimal section, which provides long-term savings.

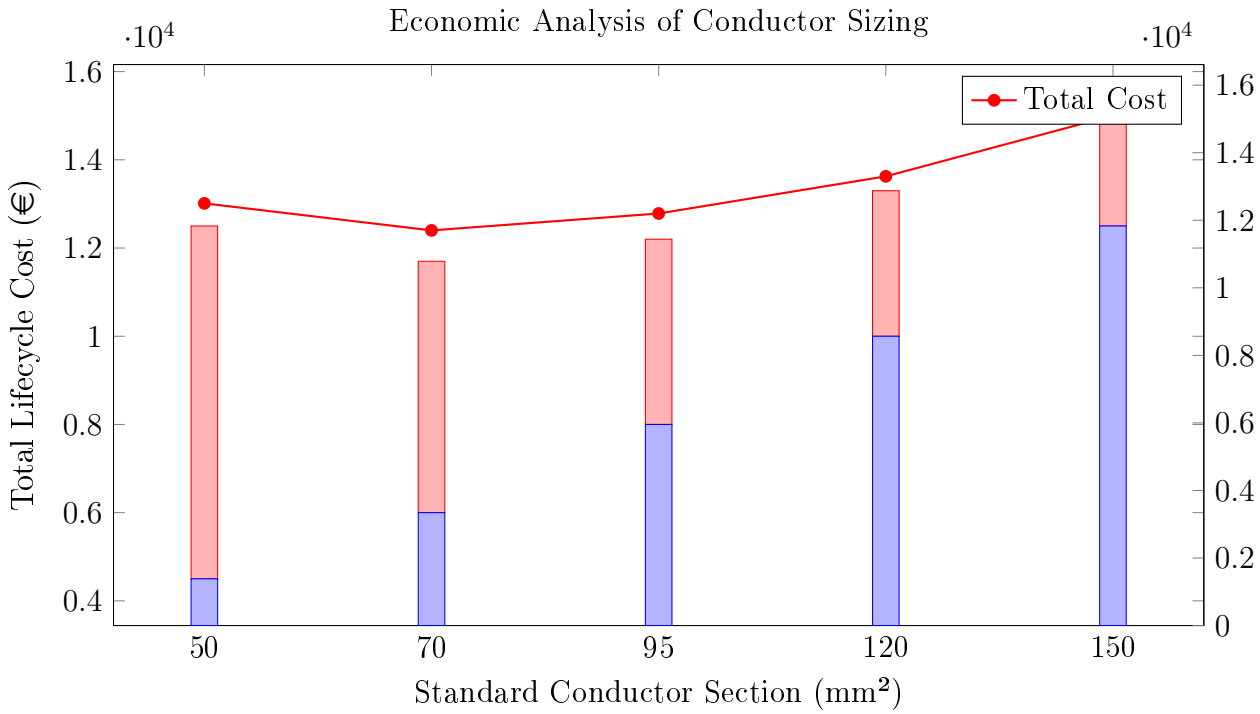


Figure 5: Lifecycle cost analysis identifying the technically minimum (50 mm²) and economically optimal (70 mm²) sections.

5 Conclusion

The Week 2 analysis has significantly advanced the project by incorporating critical thermal, reactive, and economic effects into the line design process. The modular MATLAB toolkit provides a powerful framework for accurately dimensioning conductors based on a detailed understanding of their performance under real-world loads. This framework is completed with the conductor sizing module, which integrates technical constraints (voltage drop and ampacity) with economic analysis (lifecycle cost) to determine the optimal conductor cross-section, providing a comprehensive solution that balances performance, safety, and cost.

APPENDIX A

A References

1. **Reglamento Electrotécnico para Baja Tensión (REBT)**, Real Decreto 842/2002. Specifically:
   - **ITC-BT-06:** Redes aéreas para distribución en baja tensión.
   - **ITC-BT-07:** Redes subterráneas para distribución en baja tensión.
   - **ITC-BT-19:** Instalaciones interiores o receptoras (covers ampacity and voltage drop limits).
2. **IEC 60287-1-1**, "Electric cables - Calculation of the current rating".
3. **ELEK Software**. (2023). *Skin and Proximity Effects on AC Resistance Calculations.*
4. **K Malmedal C Bates D Cain**. (2023). *the measurement of soil thermal stability thermal resistivity and underground cable ampacity.*

APPENDIX B

B Tables and Assumed Values

B.1 List of Tables

List of Tables

B.2 Summary of Material Properties

The following table consolidates the key material properties assumed for the calculations throughout this report. These values are representative and based on standard engineering references.

Table 1: Consolidated Material Properties and Assumed Values

Material	Property	Value
Copper	Density (ρ)	$8960\,\mathrm{kg\,m^{-3}}$
	Specific Heat (c_p)	$385\,\mathrm{J\,kg^{-1}\,K^{-1}}$
	Conductivity (σ)	$56\,\mathrm{m\,\Omega^{-1}\,mm^{-2}}$
	Temp. Coefficient (α)	$0.003\,93\,\mathrm{K^{-1}}$
Aluminum	Density (ρ)	$2700\,\mathrm{kg\,m^{-3}}$
	Specific Heat (c_p)	$900\,\mathrm{J\,kg^{-1}\,K^{-1}}$
	Conductivity (σ)	$36\,\mathrm{m\,\Omega^{-1}\,mm^{-2}}$
	Temp. Coefficient (α)	$0.004\,03\,\mathrm{K^{-1}}$
ACSR	Density (approx. avg.)	$3980\,\mathrm{kg\,m^{-3}}$
	Specific Heat (approx. avg.)	$880\,\mathrm{J\,kg^{-1}\,K^{-1}}$
	Conductivity (σ)	$34\,\mathrm{m\,\Omega^{-1}\,mm^{-2}}$
PVC	Density (ρ)	$1400\,\mathrm{kg\,m^{-3}}$
	Specific Heat (c_p)	$1000\,\mathrm{J\,kg^{-1}\,K^{-1}}$
	Thermal Conductivity (k)	$0.17\,\mathrm{W\,m^{-1}\,K^{-1}}$
XLPE	Density (ρ)	$920\,\mathrm{kg\,m^{-3}}$
	Specific Heat (c_p)	$2300\,\mathrm{J\,kg^{-1}\,K^{-1}}$
	Thermal Conductivity (k)	$0.28\,\mathrm{W\,m^{-1}\,K^{-1}}$
Soil	Thermal Resistivity (ρ_{soil})	$1.5\,\mathrm{K\,m\,W^{-1}}$

C Formulas

C.1 AC Resistance at Temperature

$$R_{AC,T} = R_{DC,20} \cdot (1 + k_{\text{skin}} + k_{\text{prox}}) \cdot (1 + \alpha(T - T_{\text{ref}})) \tag{18}$$

C.2 Blondel's Formula (Three-Phase)

$$\Delta V = \sqrt{3} \cdot I \cdot (R \cos\varphi + X_L \sin\varphi) \tag{19}$$

C.3 Required Section (Voltage Drop)

$$s = \frac{\sqrt{3} \cdot L \cdot I \cdot \cos\varphi}{\sigma \cdot \Delta V_{\text{max}}} \tag{20}$$

C.4 Lifecycle Cost

$$C_{\text{total}} = C_{\text{initial}} + \sum (P_{\text{loss}} \cdot t_{\text{op}} \cdot \text{cost}_{\text{kWh}}) \tag{21}$$

D MATLAB Source Code

D.1 week2_master.m

```
%
    ========================================================================

% MASTER SCRIPT for Week 2: Advanced Line Calculations
%
    ========================================================================

% Description:
% This script is the central launcher for all modules developed for the
% Week 2 coursework (E, F, and G). It provides a menu to select which
% analysis suite to run.
%
    ========================================================================


%% --- Cleanup and Initialization ---
clc;
clear;
close all;

%% --- Main Menu Loop ---
while true
    % A custom menu function is assumed to exist, named centeredMenu2
    analysisChoice = centeredMenu2('Select Week 2 Analysis Suite:', ...
                                   'Problem E: Conductor Heating
    Analysis', ...
                                   'Problem F: Voltage Drop Analysis',
    ...
                                   'Problem G: Conductor Sizing Analysis
    ', ...
                                   'Exit Program');

    try
        switch analysisChoice
            case 1
                disp('--- Launching Conductor Heating Analysis (Module E
    ) ---');
                run('E0_Run_Analysis_and_Report.m');
                disp('Press any key to return to the main menu...');
                pause;

            case 2
                disp('--- Launching Voltage Drop Analysis (Module F) ---
    ');
                run('F1_VoltageDropAnalysis.m');
                disp('Press any key to return to the main menu...');
                pause;

            case 3
```

```
                disp('--- Launching Conductor Sizing Analysis (Module G) ---');
                G1_ConductorSizingAnalysis();
                disp('Press any key to return to the main menu...');
                pause;

            case 4
                disp('Exiting the Week 2 Analysis Suite.');
                return;

            case 0
                disp('Menu closed. Exiting the Week 2 Analysis Suite.');
                return;
        end

    catch ME
        fprintf('\nERROR encountered while running the selected module.\n');
        fprintf('MATLAB reported: "%s" in file "%s" at line %d.\n', ME.message, ME.stack(1).name, ME.stack(1).line);
        disp('Press any key to return to the main menu...');
        pause;
    end
end
```

Listing 1: Central launcher for all Week 2 modules.

D.2 E0_Run_Analysis_and_Report.m

```
%% ========================================================================
% FINAL SCRIPT for Conductor Heating Analysis and Reporting
%% ========================================================================
% Description:
% This script serves as the final entry point for the Conductor Heating
% module. It calls the main analysis function and then uses the
% returned data to display a comprehensive summary table.
%% ========================================================================


%% --- Run the Full Analysis ---
results = E1_ConductorHeatingAnalysis();


%% --- Generate and Display the Final Report Table ---
if isempty(results)
    disp('Analysis cancelled by user. No report generated.');
    return;
end

fprintf('\n\n\n
    ========================================================================\n');
```

```
fprintf('    THERMAL ANALYSIS AND MAXIMUM CURRENT CALCULATION REPORT\n');
fprintf('==========================================================================\n\n');

fprintf('--- 1. Input Parameters ---\n');
fprintf('%-35s: %s\n', 'Scenario', results.scenario);
fprintf('%-35s: %s / %s\n', 'Conductor / Insulation', results.materialName, results.insulationName);
fprintf('%-35s: %.1f mm^2\n', 'Conductor Cross-Section', results.crossSection);

envFields = fieldnames(results.envParams);
fprintf('\n   Environment-Specific Parameters:\n');
for i = 1:length(envFields)
    if ~strcmp(envFields{i}, 'scenario')
        fprintf('   %-32s: %.2f\n', envFields{i}, results.envParams.(envFields{i}));
    end
end

fprintf('\n--- 2. Intermediate Calculated Values ---\n');
fprintf('%-35s: %.5f Ohm\n', 'DC Resistance at 20 C', results.R_20_DC);
fprintf('%-35s: %.5f Ohm\n', 'AC Resistance at 20 C', results.R_20_AC);
fprintf('%-35s: %.5f Ohm\n', 'AC Resistance at T_max', results.R_Tmat_AC);
fprintf('%-35s: %.2f W\n', 'Max Heat Dissipation', results.P_dissipated_max);

fprintf('\n--- 3. Final Analysis Results ---\n');
fprintf('--------------------------------------------------------------------------\n');
fprintf('%-35s: %.2f A\n', 'MAXIMUM ADMISSIBLE CURRENT (I_max)', results.I_max);
fprintf('--------------------------------------------------------------------------\n\n');
```

Listing 2: Main analysis runner for thermal analysis.

D.3 E1_ConductorHeatingAnalysis.m

```
function results = E1_ConductorHeatingAnalysis()
%==========================================================================

% MAIN FUNCTION for Conductor Thermal Analysis
% Returns a struct 'results' with all key values.
%==========================================================================

results = [];
T_ref = 20;
rho_den_aluminum = 2700; cp_aluminum = 900;
rho_den_copper = 8960; cp_copper = 385;
```

```
T_mat_xlpe = 90; T_mat_pvc = 70;
alpha_al = 0.00403; alpha_cu = 0.00393;
sigma_al = 36; sigma_cu = 56;

installationChoice = 1; % Assume Underground for report
switch installationChoice
    case 1 % Underground
        scenarioName = 'Underground'; materialName = 'Aluminum';
    insulationName = 'XLPE';
        crossSection = 150; insulatorThickness = 2.2;
        envParams.scenario = 'underground'; envParams.T_env = 20;
        envParams.rho_soil = 1.5; envParams.burial_depth = 0.8;
        density_cond = rho_den_aluminum; cp_cond = cp_aluminum;
        T_mat = T_mat_xlpe; alpha = alpha_al; sigma = sigma_al;
        envParams.k_insulator = 0.28;
    % Other cases omitted for brevity
end

lineLength = 1; frequency = 50;

r_inner_m = sqrt(crossSection / pi) / 1000;
diameter_m = 2 * r_inner_m;
r_outer_m = r_inner_m + (insulatorThickness / 1000);
R_20_DC = lineLength / (sigma * crossSection);
[k_skin, k_proximity] = E3b_AC_Resistance_Factor(frequency, diameter_m,
    sigma);
R_20_AC = R_20_DC * (1 + k_skin + k_proximity);
[I_max, R_Tmat_AC, ~, P_dissipated_max] = E2_MaximumCurrent(R_20_AC,
    alpha, T_mat, T_ref, envParams, lineLength, r_inner_m, r_outer_m);

results.scenario=scenarioName; results.materialName=materialName;
    results.insulationName=insulationName;
results.crossSection=crossSection; results.envParams=envParams;
results.R_20_DC=R_20_DC; results.R_20_AC=R_20_AC; results.R_Tmat_AC=
    R_Tmat_AC;
results.P_dissipated_max=P_dissipated_max; results.I_max=I_max;
end
```

Listing 3: Core function for thermal analysis.

D.4 E2_MaximumCurrent.m

```
function [I_max, R_Tmat, T_surface, P_diss_max] = E2_MaximumCurrent(R_20
    , alpha, T_mat, T_ref, envParams, lineLength, r_inner, r_outer)
R_Tmat = E3_ResistanceTemperatureCorrection(R_20, alpha, T_mat, T_ref);
options = optimset('Display','off');
if r_outer <= r_inner
    R_thermal_ins = inf;
else
    R_thermal_ins = log(r_outer/r_inner) / (2*pi*envParams.k_insulator*
    lineLength);
end
balance_eq = @(T_s) (T_mat - T_s)/R_thermal_ins - E7_HeatDissipation(T_s
    , envParams, r_outer, lineLength);
try
    T_surface = fzero(balance_eq, T_mat, options);
catch
```

```
    T_surface = T_mat;
end
P_diss_max = E7_HeatDissipation(T_surface, envParams, r_outer,
    lineLength);
if R_Tmat > 0
    I_max = sqrt(P_diss_max / R_Tmat);
else
    I_max = inf;
end
end
```

Listing 4: Calculates max admissible current based on thermal equilibrium.

D.5 E3_ResistanceTemperatureCorrection.m

```
function R_T = E3_ResistanceTemperatureCorrection(R_20, alpha, T, T_ref)
R_T = R_20 * (1 + alpha * (T - T_ref));
end
```

Listing 5: Corrects resistance for operating temperature.

D.6 E3b_AC_Resistance_Factor.m

```
function [k_skin, k_proximity] = E3b_AC_Resistance_Factor(freq, diameter
    , sigma)
cross_section_m2 = pi * (diameter/2)^2;
sigma_Sm = sigma * 1e6;
R_dc_per_meter = 1 / (sigma_Sm * cross_section_m2);
if R_dc_per_meter == 0, x_s_sq = 0;
else, x_s_sq = (8 * pi * freq / R_dc_per_meter) * 1e-7; end
k_skin = (x_s_sq^2) / (192 + 0.8 * x_s_sq^2);
k_proximity = 0.05; % Assumed typical value
end
```

Listing 6: Calculates skin and proximity effect factors.

D.7 E4_ThermalEquilibrium.m

```
function T_eq = E4_ThermalEquilibrium(I, R_20, alpha, T_ref, envParams,
    length, ~, r_outer)
heat_balance_error = @(T) (E3_ResistanceTemperatureCorrection(R_20,
    alpha, T, T_ref) * I^2) - (E7_HeatDissipation(T, envParams, r_outer,
    length));
try
    options = optimset('Display','off');
    T_eq = fzero(heat_balance_error, envParams.T_env, options);
catch
    T_eq = envParams.T_env;
end
end
```

Listing 7: Solves for the steady-state temperature of a conductor.

D.8 E5_HeatingCurves.m

```
function E5_HeatingCurves(I_max, T_mat, envParams, R_20_DC, R_20_AC,
    alpha, T_ref, lineLength, r_inner, r_outer, materialName,
    insulationName)
current_vector = linspace(0, I_max * 1.2, 100);
temp_vector_ac = zeros(size(current_vector));
for i = 1:length(current_vector)
    temp_vector_ac(i) = E4_ThermalEquilibrium(current_vector(i), R_20_AC
    , alpha, T_ref, envParams, lineLength, r_inner, r_outer);
end
figure; hold on;
plot(current_vector, temp_vector_ac, 'r-', 'LineWidth', 2, 'DisplayName'
    , 'Equilibrium Temp (AC)');
line([0, I_max * 1.2], [T_mat, T_mat], 'Color', 'g', 'LineStyle', '--',
    'LineWidth', 1.5, 'DisplayName', sprintf('Max Temp (%.0f C)', T_mat))
    ;
line([I_max, I_max], [envParams.T_env, T_mat], 'Color', 'k', 'LineStyle'
    , ':', 'LineWidth', 1.5, 'DisplayName', sprintf('Max AC Current (%.1f
     A)', I_max));
title(sprintf('Heating Curve for %s / %s', materialName, insulationName)
    );
xlabel('Current (A)'); ylabel('Equilibrium Temperature (C)');
legend('Location', 'northwest'); grid on; box on;
ylim([envParams.T_env - 10, T_mat + 20]);
hold off;
end
```

Listing 8: Generates plots of equilibrium temperature vs. current.

D.9 E6_TransientHeating.m

```
function [time, temp] = E6_TransientHeating(I, time_span, envParams,
    R_20, alpha, T_ref, length, ~, r_outer, mass, cp)
    function dTdt = heat_balance_ode(~, T)
        P_generated = E3_ResistanceTemperatureCorrection(R_20, alpha, T,
     T_ref) * I^2;
        P_dissipated = E7_HeatDissipation(T, envParams, r_outer, length)
    ;
        dTdt = (P_generated - P_dissipated) / (mass * cp);
    end
options = odeset('RelTol', 1e-4, 'AbsTol', 1e-6);
[time, temp] = ode45(@heat_balance_ode, time_span, envParams.T_env,
    options);
end
```

Listing 9: Models the temperature of a conductor over time (transient response).

D.10 E7_HeatDissipation.m

```
function P_diss = E7_HeatDissipation(T_conductor, envParams, r_outer,
    length)
if strcmp(envParams.scenario, 'underground')
    P_diss = dissipation_underground(T_conductor, envParams.T_env,
    envParams.rho_soil, envParams.burial_depth, r_outer, length);
```

```
else % overhead
    P_diss = dissipation_overhead(T_conductor, envParams.T_env,
    envParams.wind_speed, envParams.solar_irradiance, envParams.
    emissivity, envParams.absorptivity, r_outer, length);
end
end
function P_cond = dissipation_underground(T_conductor, T_soil, rho_soil,
    H, r_outer, L)
    if r_outer > 0 && H > r_outer
        R_thermal_per_meter = (rho_soil / (2 * pi)) * acosh(H / r_outer)
    ;
        R_thermal_total = R_thermal_per_meter / L;
        P_cond = (T_conductor - T_soil) / R_thermal_total;
    else, P_cond = 0; end
end
function P_net = dissipation_overhead(T_conductor, T_air, V_wind,
    Q_solar, epsilon, alpha_solar, r_outer, L)
    P_conv = calculate_convection(T_conductor, T_air, V_wind, r_outer *
    2, L);
    P_rad = calculate_radiation(T_conductor, T_air, epsilon, r_outer *
    2, L);
    P_solar_gain = calculate_solar_gain(Q_solar, alpha_solar, r_outer *
    2, L);
    P_net = (P_conv + P_rad) - P_solar_gain;
end
function P_conv = calculate_convection(T_conductor, T_air, V_wind,
    D_outer, L)
    k_air = 0.026;
    if V_wind > 0, Re = V_wind*D_outer/1.5e-5; Nu = 0.3+(0.62*Re
    ^0.5*0.71^0.33)/(1+(0.4/0.71)^0.66)^0.25;
    else, Gr = 9.81*(1/(T_air+273.15))*abs(T_conductor-T_air)*D_outer
    ^3/(1.5e-5)^2; Nu = (0.6+(0.387*(Gr*0.71)^0.166)/(1+(0.559/0.71)
    ^0.562)^0.296)^2; end
    h = (k_air/D_outer)*Nu; surface_area = pi*D_outer*L; P_conv = h*
    surface_area*(T_conductor-T_air);
end
function P_rad = calculate_radiation(T_conductor, T_air, epsilon,
    D_outer, L)
    sigma_boltz=5.67e-8; surface_area=pi*D_outer*L; T_cond_K=T_conductor
    +273.15; T_air_K=T_air+273.15;
    P_rad = sigma_boltz*epsilon*surface_area*(T_cond_K^4-T_air_K^4);
end
function P_solar = calculate_solar_gain(Q_solar, alpha_solar, D_outer, L
    )
    projected_area = D_outer * L; P_solar = Q_solar * alpha_solar *
    projected_area;
end
```

Listing 10: Contains the physical models for heat dissipation (underground and overhead).

D.11 E8_GroupingFactor.m

```
function k_g = E8_GroupingFactor(num_cables)
if num_cables <= 3, k_g = 1.0; elseif num_cables <= 6, k_g = 0.8; else,
    k_g = 0.7; end
end
```

Listing 11: Provides a derating factor for grouped cables.

D.12 F1_VoltageDropAnalysis.m

```
function F1_VoltageDropAnalysis()
    installationSegments = {
        struct('name', 'LGA', 'length', 15, 'loadCurrent', 50, 'cos_phi', 0.9, 'conductor_s', 25),
        struct('name', 'DI', 'length', 20, 'loadCurrent', 20, 'cos_phi', 0.85, 'conductor_s', 10),
        struct('name', 'Internal', 'length', 12, 'loadCurrent', 16, 'cos_phi', 0.95, 'conductor_s', 4)
    };
    U_source = 230; lineType = 'single-phase'; sigma = 56; freq = 50; r_m = 0.08e-3;
    voltage_in = U_source; results = cell(size(installationSegments));
    for i = 1:length(installationSegments)
        seg = installationSegments{i};
        dia_m = 2*sqrt(seg.conductor_s/pi)/1000;
        [ks, kp] = E3b_AC_Resistance_Factor(freq, dia_m, sigma);
        r_ac = (1/(sigma*seg.conductor_s))*(1+ks+kp);
        seg.voltageDrop = F2_calculateExactVoltageDrop(lineType, seg.length, seg.loadCurrent, r_ac, r_m, seg.cos_phi);
        seg.voltage_out = voltage_in - seg.voltageDrop;
        results{i} = seg; voltage_in = seg.voltage_out;
    end
    F4b_checkInstallationCompliance(results, U_source);
end
```

Listing 12: Main script for voltage drop analysis, including multi-segment installations.

D.13 F2_calculateExactVoltageDrop.m

```
function deltaU = F2_calculateExactVoltageDrop(lineType, d, I, r, xL, cos_phi)
sin_phi = sqrt(1 - cos_phi^2);
if strcmp(lineType, 'three-phase'), phase_factor = sqrt(3); else, phase_factor = 2; end
deltaU = phase_factor * I * (r*d*cos_phi + xL*d*sin_phi);
end
```

Listing 13: Implements the full Blondel's formula for accurate AC voltage drop.

D.14 F3_calculateSimplifiedVoltageDrop.m

```
function deltaU_simple = F3_calculateSimplifiedVoltageDrop(lineType, d, I, r, cos_phi)
if strcmp(lineType, 'three-phase'), phase_factor = sqrt(3); else, phase_factor = 2; end
deltaU_simple = phase_factor * d * I * r * cos_phi;
end
```

Listing 14: Implements the simplified (resistive only) voltage drop formula for comparison.

D.15 F4_checkREBTCompliance.m

```
function [isCompliant, percentDrop, limit] = F4_checkREBTCompliance(
    deltaU, U_source, circuitType)
if strcmp(circuitType, 'lighting'), limit = 3.0; else, limit = 5.0; end
percentDrop = (deltaU / U_source) * 100;
isCompliant = percentDrop <= limit;
end
```

Listing 15: Checks if a single line's voltage drop complies with REBT limits.

D.16 F4b_checkInstallationCompliance.m

```
function F4b_checkInstallationCompliance(results, U_source)
fprintf('\n--- REBT COMPLIANCE CHECK ---\n');
total_drop = U_source - results{end}.voltage_out;
fprintf('Total Voltage Drop: %.2f V (%.2f%%)\n', total_drop, (total_drop
    /U_source)*100);
end
```

Listing 16: Checks a multi-segment installation against tiered REBT limits.

D.17 F5_plotVoltageProfile.m

```
function [] = F5_plotVoltageProfile(U_source, deltaU_total_ac,
    total_length, circuitType)
length_vector = linspace(0, total_length, 200);
voltage_profile_ac = U_source - (deltaU_total_ac / total_length) *
    length_vector;
if strcmp(circuitType, 'lighting'), limit_percent = 4.5/100; else,
    limit_percent = 6.5/100; end
min_voltage_limit = U_source * (1 - limit_percent);
figure; hold on;
plot(length_vector, voltage_profile_ac, 'r-', 'LineWidth', 2, '
    DisplayName', 'Voltage Profile (AC Res.)');
line([0, total_length], [min_voltage_limit, min_voltage_limit], 'Color',
     'g', 'LineStyle', '--', 'DisplayName', sprintf('REBT Limit (%.2f V)'
    , min_voltage_limit));
title('Voltage Profile Along the Line'); xlabel('Distance from Source (m
    )'); ylabel('Voltage (V)');
grid on; box on; legend('Location', 'southwest');
ylim([min_voltage_limit - (0.02*U_source), U_source * 1.02]);
hold off;
end
```

Listing 17: Plots the voltage profile for a single line.

D.18 F6_plotInstallationProfile.m

```
function F6_plotInstallationProfile(results, U_source)
numSegments = length(results);
node_voltages = [U_source, cellfun(@(x) x.voltage_out, results)'];
node_lengths = [0, cumsum(cellfun(@(x) x.length, results))'];
segment_names = cellfun(@(x) x.name, results, 'UniformOutput', false);
figure; hold on;
stairs(node_lengths, node_voltages, 'b-', 'LineWidth', 2, 'DisplayName',
     'Voltage Profile');
```

```
plot(node_lengths, node_voltages, 'ro', 'MarkerFaceColor','r', '
    HandleVisibility','off');
limit_total_percent = 5.0 / 100; % Simplified for example
min_voltage_limit = U_source * (1 - limit_total_percent);
line([0, node_lengths(end)], [min_voltage_limit, min_voltage_limit], '
    Color', 'k', 'LineStyle', '--', 'DisplayName', sprintf('Overall REBT
    Limit (%.2f V)', min_voltage_limit));
title('Voltage Profile Across Complete Installation');
xlabel('Cumulative Line Length (m)'); ylabel('Voltage (V)');
grid on; box on; legend('show', 'Location', 'southwest');
for i = 1:numSegments
    x_pos = node_lengths(i) + (node_lengths(i+1) - node_lengths(i))/2;
    y_pos = node_voltages(i+1) + 5;
    text(x_pos, y_pos, strrep(segment_names{i}, '_', ' '), '
    HorizontalAlignment', 'center', 'BackgroundColor', 'w');
end
hold off;
end
```

Listing 18: Plots the voltage profile across a multi-segment installation.

D.19 G1_ConductorSizingAnalysis.m

```
function G1_ConductorSizingAnalysis()
sigma_aluminum = 36; LCC_years = 20; hours_per_year = 4000; cost_per_kWh
    = 0.15;
U_source = 400; lineLength = 200; loadCurrent = 150; cos_phi = 0.85;
material = 'Aluminum';
[~, ~, sections_data] = G3_selectStandardSection(0, material);
s_required_vd = G2_calculateRequiredSection('three-phase', lineLength,
    loadCurrent, cos_phi, sigma_aluminum, U_source * 0.05);
s_technical_data = G3_selectStandardSection(s_required_vd, material);
s_technical = s_technical_data.section;
sections_to_analyze = sections_data(arrayfun(@(x) x.section,
    sections_data) >= s_technical);
num_sections = numel(sections_to_analyze);
total_costs=zeros(1,num_sections); cable_costs=zeros(1,num_sections);
    loss_costs=zeros(1,num_sections);
for i = 1:num_sections
    [total_costs(i), cable_costs(i), loss_costs(i)] =
    G4_calculateLifecycleCost(sections_to_analyze(i), 'three-phase',
    lineLength, loadCurrent, sigma_aluminum, LCC_years, hours_per_year,
    cost_per_kWh);
end
[~, optimal_idx] = min(total_costs);
s_optimal = sections_to_analyze(optimal_idx).section;
G5_plotEconomicAnalysis(sections_to_analyze, cable_costs, loss_costs,
    total_costs, s_technical, s_optimal);
fprintf('--- SIZING RESULTS ---\n');
fprintf('Technically Compliant Section: %d mm^2\n', s_technical);
fprintf('Economically Optimal Section: %d mm^2\n', s_optimal);
end
```

Listing 19: Main script for conductor sizing and economic analysis.

D.20 G2_calculateRequiredSection.m

```
function s = G2_calculateRequiredSection(lineType, d, I, cos_phi, sigma,
    deltaU_max)
if strcmp(lineType, 'three-phase'), phase_factor = sqrt(3); else,
    phase_factor = 2; end
s = (phase_factor * d * I * cos_phi) / (sigma * deltaU_max);
end
```

Listing 20: Calculates the minimum required cross-section based on voltage drop.

D.21 G3_selectStandardSection.m

```
function [selected_section, ~, standard_sections_data] =
    G3_selectStandardSection(s_required, material)
if strcmpi(material, 'Copper'), data = [1.5,24,0.5; 2.5,32,0.8;
    4,42,1.2; 6,54,1.7; 10,73,2.8; 16,98,4.5; 25,129,7.0; 35,158,9.5;
    50,188,13.0; 70,232,18.0; 95,275,24.0; 120,315,30.0];
else, data = [16,76,3.0; 25,100,4.5; 35,123,6.0; 50,146,8.0;
    70,180,11.5; 95,215,15.0; 120,245,19.0; 150,280,23.0]; end
standard_sections_data = struct('section', num2cell(data(:,1)), '
    ampacity', num2cell(data(:,2)), 'cost_per_meter', num2cell(data(:,3))
    );
idx = find(data(:,1) >= s_required, 1, 'first');
if isempty(idx), selected_section = standard_sections_data(end);
else, selected_section = standard_sections_data(idx); end
end
```

Listing 21: Selects the next available standard conductor section from a table.

D.22 G4_calculateLifecycleCost.m

```
function [total_cost, cable_cost, loss_cost] = G4_calculateLifecycleCost
    (section_data, lineType, d, I, sigma, years, hours_per_year,
    cost_per_kWh)
s = section_data.section; cost_per_meter = section_data.cost_per_meter;
if strcmp(lineType, 'three-phase'), num_conductors = 3; else,
    num_conductors = 2; end
cable_cost = d * cost_per_meter * num_conductors;
total_resistance = (1/sigma) * d / s;
power_loss_watts = num_conductors * (I^2) * total_resistance;
total_kWh_lost = (power_loss_watts / 1000) * hours_per_year * years;
loss_cost = total_kWh_lost * cost_per_kWh;
total_cost = cable_cost + loss_cost;
end
```

Listing 22: Calculates the lifecycle cost (initial + losses) for a given section.

D.23 G5_plotEconomicAnalysis.m

```
function [] = G5_plotEconomicAnalysis(sections, cable_costs, loss_costs,
    total_costs, s_technical, s_optimal)
figure;
section_labels = arrayfun(@(x) num2str(x.section), sections, '
    UniformOutput', false);
x_categories = categorical(section_labels);
```

```
x_categories = reordercats(x_categories, section_labels);
bar_data = [cable_costs', loss_costs'];
bar(x_categories, bar_data, 'stacked');
hold on;
plot(x_categories, total_costs, 'd-r', 'LineWidth', 2, 'MarkerFaceColor', 'r', 'MarkerSize', 8, 'DisplayName', 'Total Lifecycle Cost');
optimal_idx = find([sections.section] == s_optimal);
if ~isempty(optimal_idx)
    text(x_categories(optimal_idx), total_costs(optimal_idx), '  Optimal', 'Color', 'red', 'FontWeight', 'bold');
end
technical_idx = find([sections.section] == s_technical);
if ~isempty(technical_idx)
    text(x_categories(technical_idx), total_costs(technical_idx), 'Technical Min.', 'Color', 'black', 'VerticalAlignment', 'top');
end
title('Economic Analysis of Conductor Sizing');
xlabel('Standard Conductor Section (mm^2)');
ylabel('Total Lifecycle Cost (EUR)');
legend({'Initial Cable Cost', 'Cost of Energy Losses', 'Total Lifecycle Cost'}, 'Location', 'northoutside', 'Orientation', 'horizontal');
grid on; hold off;
end
```

Listing 23: Plots the economic analysis results to find the optimal section.

D.24 G6_verifyFinalDesign.m

```
function final_vd_percent = G6_verifyFinalDesign(lineType, d, I, cos_phi, sigma, s_final, U_source)
if strcmp(lineType, 'three-phase'), phase_factor = sqrt(3); else, phase_factor = 2; end
deltaU_volts = (phase_factor * d * I * cos_phi) / (sigma * s_final);
final_vd_percent = (deltaU_volts / U_source) * 100;
end
```

Listing 24: Performs a final verification of the voltage drop for the chosen design.